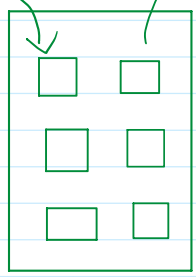
Producer-Consumer Problem

07 February 2026 17:17

Producer thread is responsible to produce items to the queue and consumer thread is responsible to consume items from the queue. If queue is empty the consumer thread will call wait() method and enter into waiting state. After producing items to the queue producer thread is responsible to call notify() method or notifyAll() method, then waiting consumer will get notification and continue its execution with updated items.

Producer Thread

```
class ProducerThread {
    produce() {
        synchronized(q) {
            produce items to queue
            q.notify();
        }
    }
}
```



Queue q

common buffer

Consumer Thread.

```
class ConsumerThread {
    consumer() {
        synchronized(q) {
            if (q.isempty()) {
                q.wait();
            }
            else {
                consume item;
            }
        }
    }
}
```

(*) Both share common buffer.

```
package MyThreads.producerconsumer;
public class MyBuffer {
    private int data;
    private boolean hasData = false;
    //producer method
    public synchronized void produce(int value){
        while(hasData){
            try{
                wait();
            }
            catch(InterruptedException e){
                e.printStackTrace();
            }
        }
```

Handles spurious wakeups,

Even in condition is not checked

```
            catch(InterruptedException e){
                e.printStackTrace();
            }
        }
        data = value;
        hasData = true;
        System.out.println("Produced: " + value);
        notify(); //notify consumer
    }
    //consumer method
    public synchronized int consume(){
        while (!hasData) {
            try {
                wait();
            } catch (InterruptedException e) {
                e.printStackTrace();
            } // wait if buffer is empty
        }
        System.out.println("Consumed: " + data);
        hasData = false;
        notify(); //Notify producer
        return data;
    }
}
```

Handles spurious wakeups,
Ensure condition is re-checked.
Best practices in multithreading.

```
package MyThreads.producerconsumer;
public class Producer extends Thread {
    private MyBuffer buffer;
    public Producer(MyBuffer buffer) {
        this.buffer = buffer;
    }

    @Override
    public void run(){
        for (int i = 100; i <= 500; i+=100) {
            buffer.produce(i);
            try{
                Thread.sleep(1000);
            }
            catch(InterruptedException ie){
                ie.printStackTrace();
            }
        }
    }
}
```

```
package MyThreads.producerconsumer;
public class Consumer extends Thread {
    private MyBuffer buffer;
    public Consumer(MyBuffer buffer) {
        this.buffer = buffer;
    }
    @Override
    public void run() {
        for (int i = 100; i <= 500; i+=100) {
            buffer.consume();
```

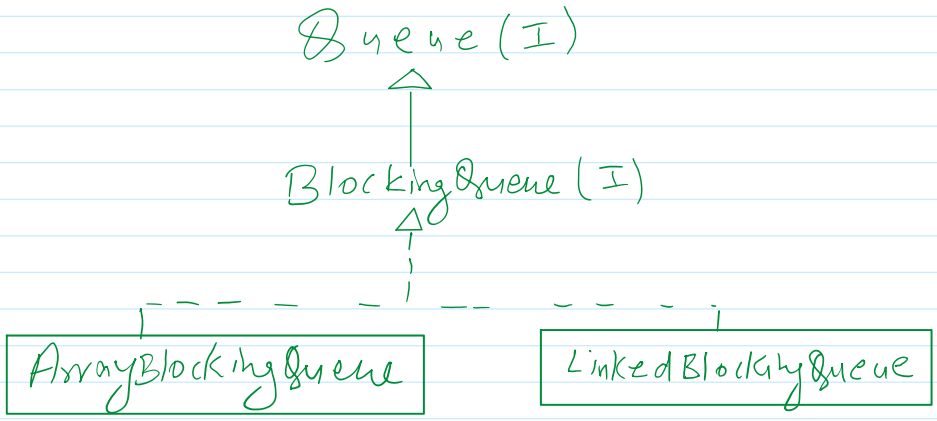
```
            try {
                Thread.sleep(1500);
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
    }
}
```

```
package MyThreads.producerconsumer;
public class Main {
    public static void main(String[] args) {
        MyBuffer buffer = new MyBuffer();
        Producer p = new Producer(buffer);
        Consumer c = new Consumer(buffer);
        p.start();
        c.start();
    }
}
```

```
package MyThreads.producerconsumer;
public class Main {
    public static void main(String[] args) {
        MyBuffer buffer = new MyBuffer();
        Producer p = new Producer(buffer);
        Consumer c = new Consumer(buffer);
        p.start();
        c.start();
    }
}
```

Output:

```
Produced: 100
Consumed: 100
Produced: 200
Consumed: 200
Produced: 300
Consumed: 300
Produced: 400
Consumed: 400
Produced: 500
Consumed: 500
```



Example using BlockingQueue

```
package MyThreads.producerconsumer;

import java.util.concurrent.BlockingQueue;
public class ProducerUsingBQ implements Runnable{
    private BlockingQueue<Integer> queue;
    public ProducerUsingBQ(BlockingQueue<Integer> queue) {
        this.queue = queue;
    }
    @Override
    public void run() {
        for (int i = 0; i < 5; i++) {
```

```
            try {
                queue.put(i+10);
                System.out.println("Produced: " + (i+10));
                Thread.sleep(1000);
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
    }
}
```

```
package MyThreads.producerconsumer;
import java.util.concurrent.BlockingQueue;
public class ConsumerUsingBQ implements Runnable{

    private BlockingQueue<Integer> queue;

    public ConsumerUsingBQ(BlockingQueue<Integer> queue) {
        this.queue = queue;
    }

    @Override
    public void run() {
        for (int i = 0; i < 5; i++) {
            try {
                int value = queue.take();  //waits if empty
                System.out.println("Consumed: " + value);
                Thread.sleep(1500);
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
    }
}
```

```
package MyThreads.producerconsumer;
import java.util.concurrent.ArrayBlockingQueue;
import java.util.concurrent.BlockingQueue;
public class MainUsingBQ {
    public static void main(String[] args) {
        BlockingQueue<Integer> queue = new ArrayBlockingQueue<>(1);
        new Thread(new ProducerUsingBQ(queue)).start();
        new Thread(new ConsumerUsingBQ(queue)).start();
    }
}
```

Output:

```
Consumed: 10
Produced: 10
Produced: 11
Consumed: 11
Produced: 12
Consumed: 12
Produced: 13
Consumed: 13
Produced: 14
Consumed: 14
```